



```
In [1]: # This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/c
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
import seaborn as sns
# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list
# the files in the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets
# deleted when you exit the kernel.
# You can also write temporary files to /kaggle/temp/, but they won't be saved
```

/kaggle/input/spotify-dataset-for-churn-analysis/spotify_churn_dataset.csv

```
In [2]: df = pd.read_csv("/kaggle/input/spotify-dataset-for-churn-analysis/spotify_churn_dataset.csv")
```

```
In [3]: df.head()
```

```
Out[3]:
```

	user_id	gender	age	country	subscription_type	listening_time	songs_played
0	1	Female	54	CA	Free	26	
1	2	Other	33	DE	Family	141	
2	3	Male	38	AU	Premium	199	
3	4	Female	22	CA	Student	36	
4	5	Other	29	US	Family	250	

```
In [4]: df.tail()
```

```
Out[4]:
```

	user_id	gender	age	country	subscription_type	listening_time	songs_played
7995	7996	Other	44	DE	Student	237	
7996	7997	Male	34	AU	Premium	61	
7997	7998	Female	17	US	Free	81	
7998	7999	Female	34	IN	Student	245	
7999	8000	Other	45	AU	Free	210	

```
In [5]: df.shape
```

```
Out[5]: (8000, 12)
```

```
In [6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   user_id                               8000 non-null   int64
1   gender                                8000 non-null   object
2   age                                    8000 non-null   int64
3   country                               8000 non-null   object
4   subscription_type                     8000 non-null   object
5   listening_time                        8000 non-null   int64
6   songs_played_per_day                  8000 non-null   int64
7   skip_rate                             8000 non-null   float64
8   device_type                           8000 non-null   object
9   ads_listened_per_week                 8000 non-null   int64
10  offline_listening                     8000 non-null   int64
11  is_churned                            8000 non-null   int64
dtypes: float64(1), int64(7), object(4)
memory usage: 750.1+ KB
```

```
In [7]: df.describe()
```

```
Out[7]:
```

	user_id	age	listening_time	songs_played_per_day	skip_ra
count	8000.00000	8000.000000	8000.000000	8000.000000	8000.0000
mean	4000.50000	37.662125	154.068250	50.127250	0.3001
std	2309.54541	12.740359	84.015596	28.449762	0.1735
min	1.00000	16.000000	10.000000	1.000000	0.0000
25%	2000.75000	26.000000	81.000000	25.000000	0.1500
50%	4000.50000	38.000000	154.000000	50.000000	0.3000
75%	6000.25000	49.000000	227.000000	75.000000	0.4500
max	8000.00000	59.000000	299.000000	99.000000	0.6000

```
In [8]: df.isnull().sum()
```

```
Out[8]: user_id      0
gender      0
age         0
country     0
subscription_type  0
listening_time  0
songs_played_per_day  0
skip_rate   0
device_type  0
ads_listened_per_week  0
offline_listening  0
is_churned   0
dtype: int64
```

```
In [9]: df.duplicated().sum()
```

```
Out[9]: 0
```

```
In [10]: df.dtypes
```

```
Out[10]: user_id      int64
gender      object
age         int64
country     object
subscription_type  object
listening_time  int64
songs_played_per_day  int64
skip_rate    float64
device_type  object
ads_listened_per_week  int64
offline_listening  int64
is_churned    int64
dtype: object
```

```
In [11]: df.columns
```

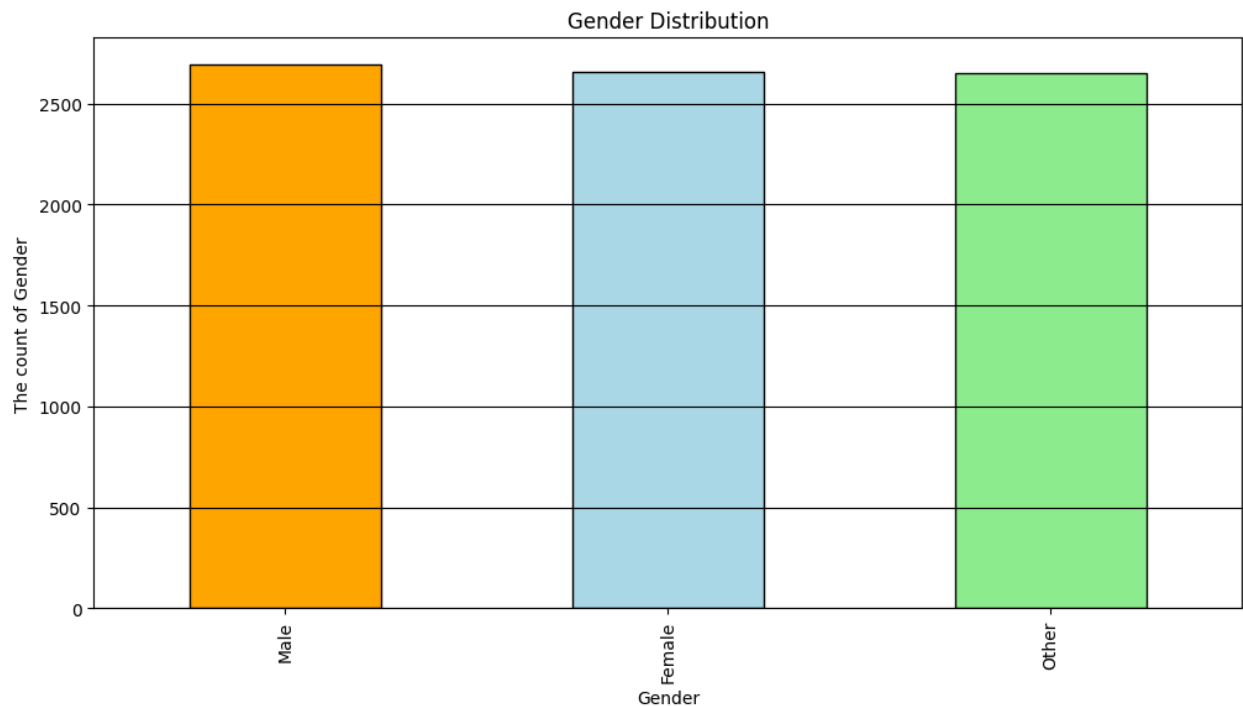
```
Out[11]: Index(['user_id', 'gender', 'age', 'country', 'subscription_type',
               'listening_time', 'songs_played_per_day', 'skip_rate', 'device_type',
               'ads_listened_per_week', 'offline_listening', 'is_churned'],
              dtype='object')
```

```
In [12]: df["gender"].value_counts()
```

```
Out[12]: gender
Male      2691
Female    2659
Other     2650
Name: count, dtype: int64
```

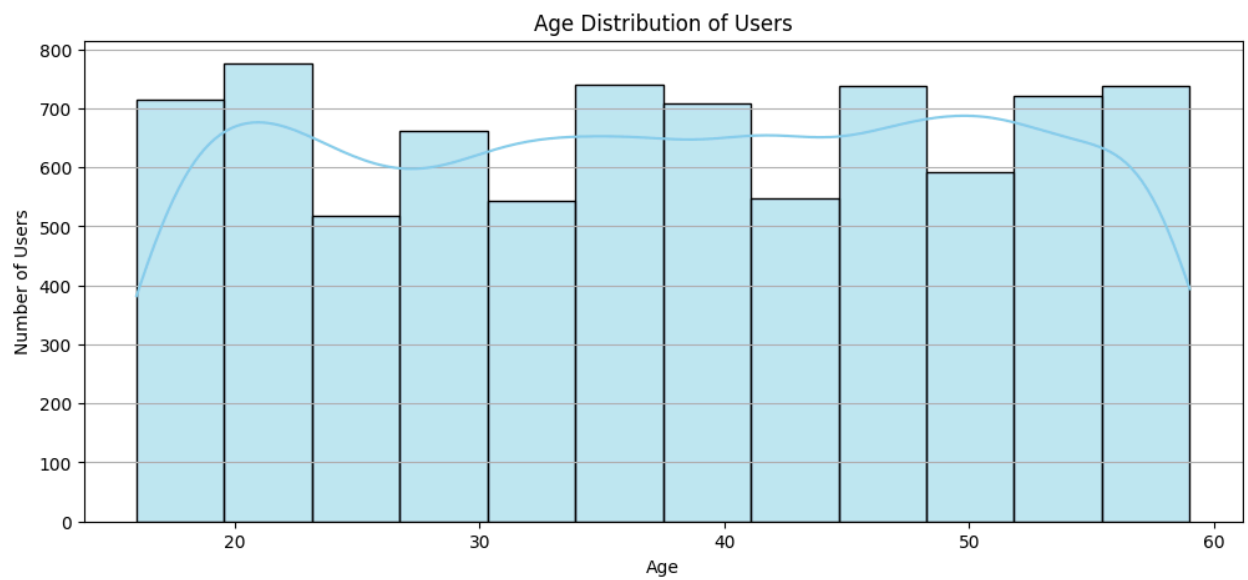
```
In [13]: plt.figure(figsize = (12,6))
df["gender"].value_counts().plot(kind = "bar", color = ["Orange", "LightBlue",
plt.title("Gender Distribution")
plt.xlabel("Gender")
```

```
plt.ylabel("The count of Gender")
plt.grid(axis = "y", color = "black")
plt.show()
```



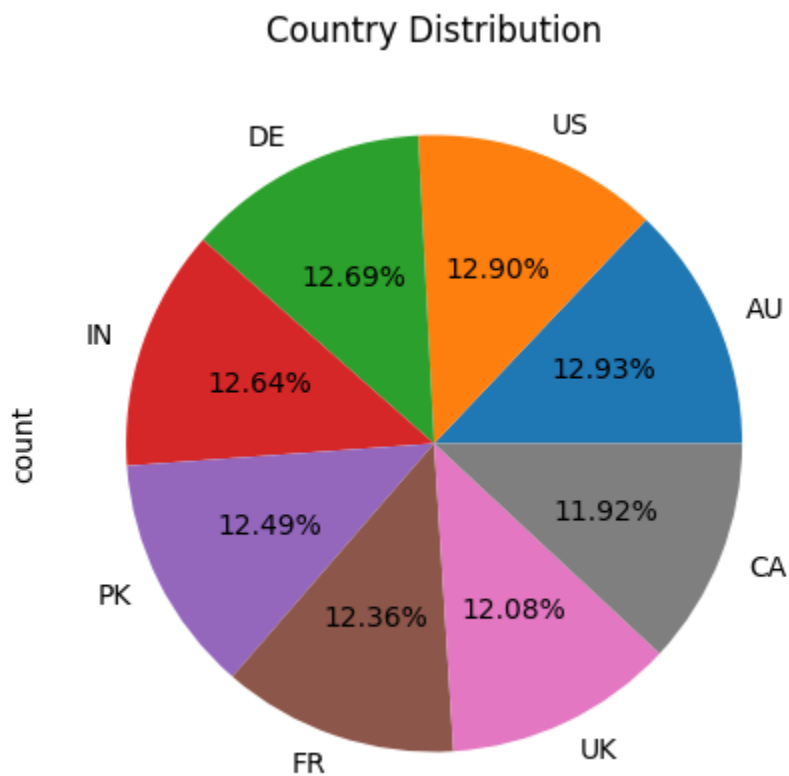
```
In [14]: plt.figure(figsize=(12,5))
sns.histplot(df["age"], bins=12, kde=True, color="skyblue", edgecolor="black")
plt.title("Age Distribution of Users")
plt.xlabel("Age")
plt.ylabel("Number of Users")
plt.grid(axis = "y")
plt.show()
```

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
 with pd.option_context('mode.use_inf_as_na', True):



```
In [15]: plt.figure(figsize=(12,5))
df["country"].value_counts().plot(kind = "pie", autopct = "%1.2f%%")
plt.title("Country Distribution")
```

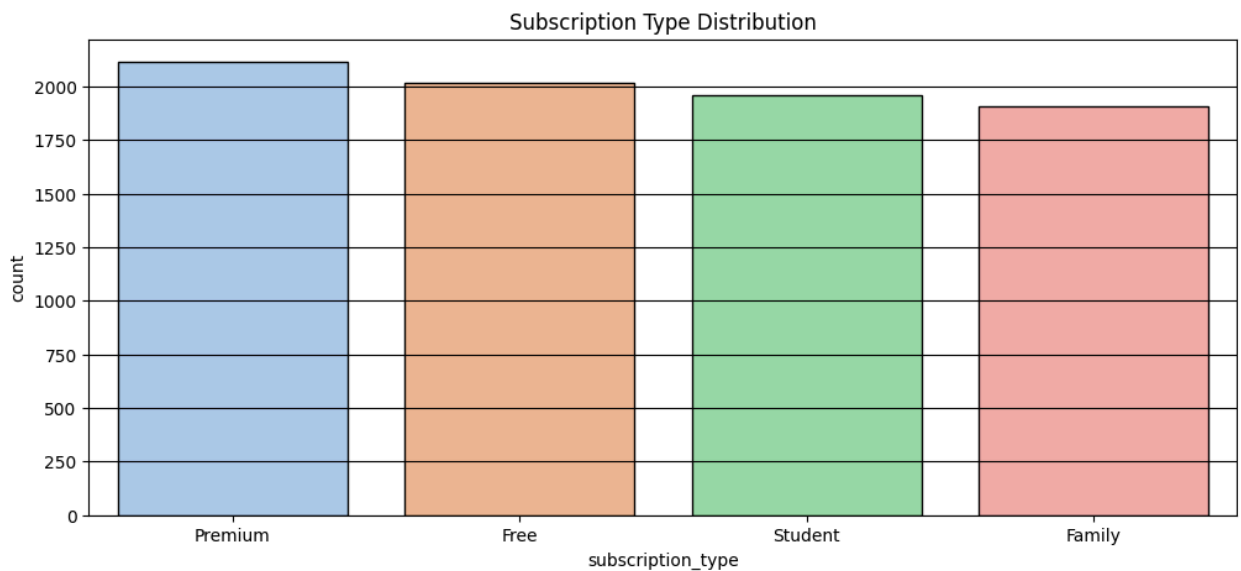
```
Out[15]: Text(0.5, 1.0, 'Country Distribution')
```



```
In [16]: df["subscription_type"].value_counts()
```

```
Out[16]: subscription_type
Premium    2115
Free       2018
Student    1959
Family     1908
Name: count, dtype: int64
```

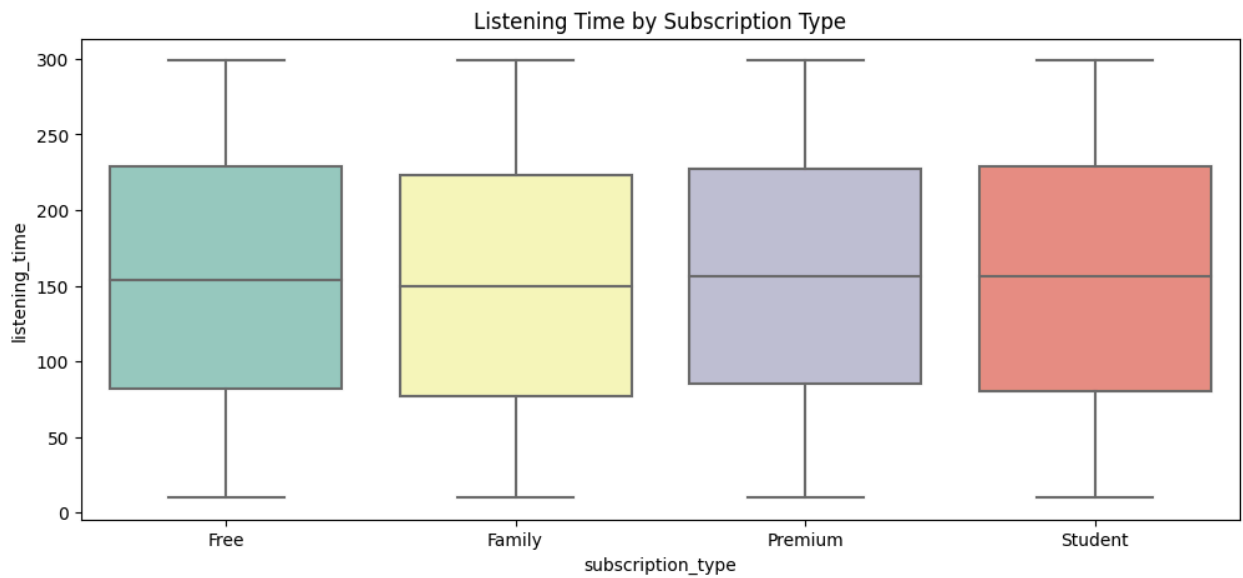
```
In [17]: plt.figure(figsize=(12,5))
sns.countplot(x="subscription_type", data=df, palette="pastel", order=df["subs
plt.title("Subscription Type Distribution")
plt.grid(axis = "y", color = "black")
plt.show()
```



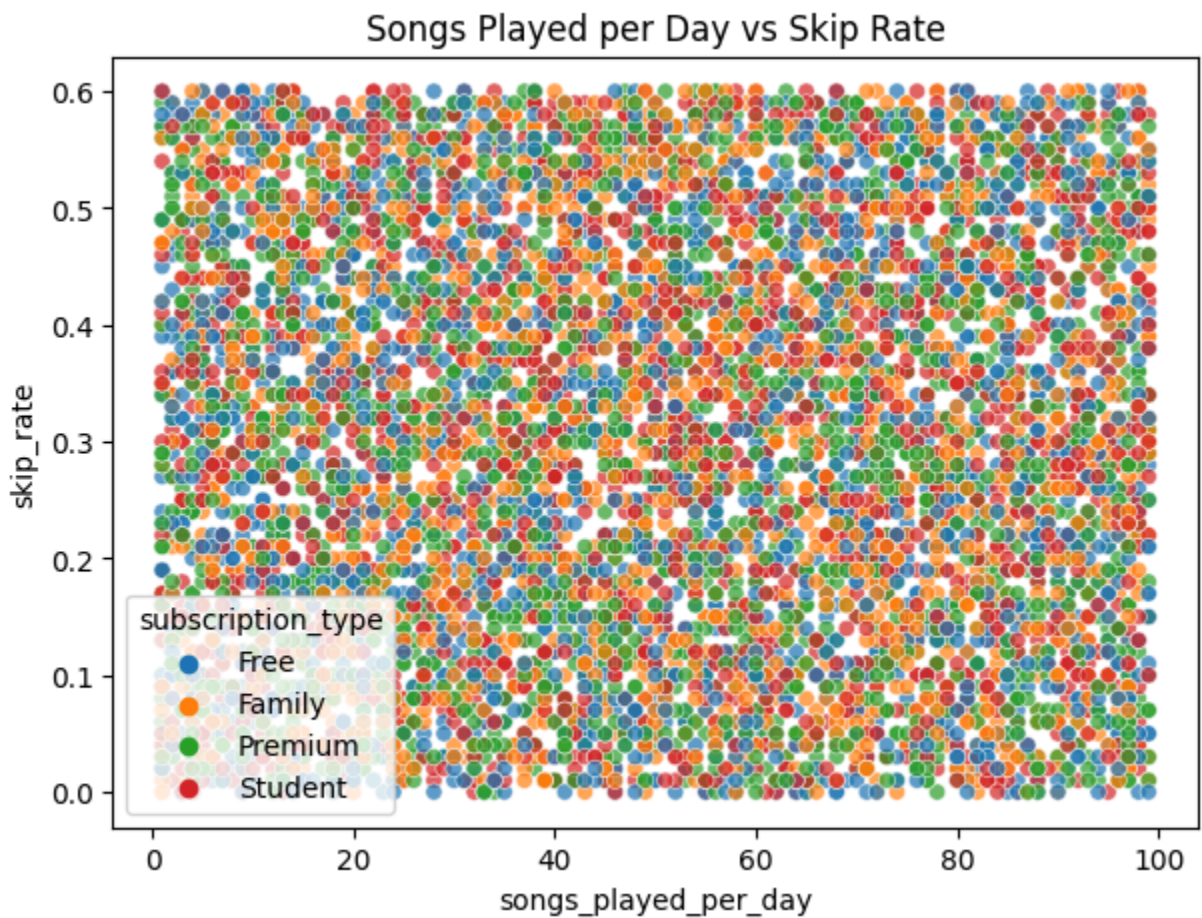
```
In [18]: df.columns
```

```
Out[18]: Index(['user_id', 'gender', 'age', 'country', 'subscription_type',
               'listening_time', 'songs_played_per_day', 'skip_rate', 'device_type',
               'ads_listened_per_week', 'offline_listening', 'is_churned'],
              dtype='object')
```

```
In [19]: plt.figure(figsize=(12,5))
sns.boxplot(x="subscription_type", y="listening_time", data=df, palette="Set3")
plt.title("Listening Time by Subscription Type")
plt.show()
```

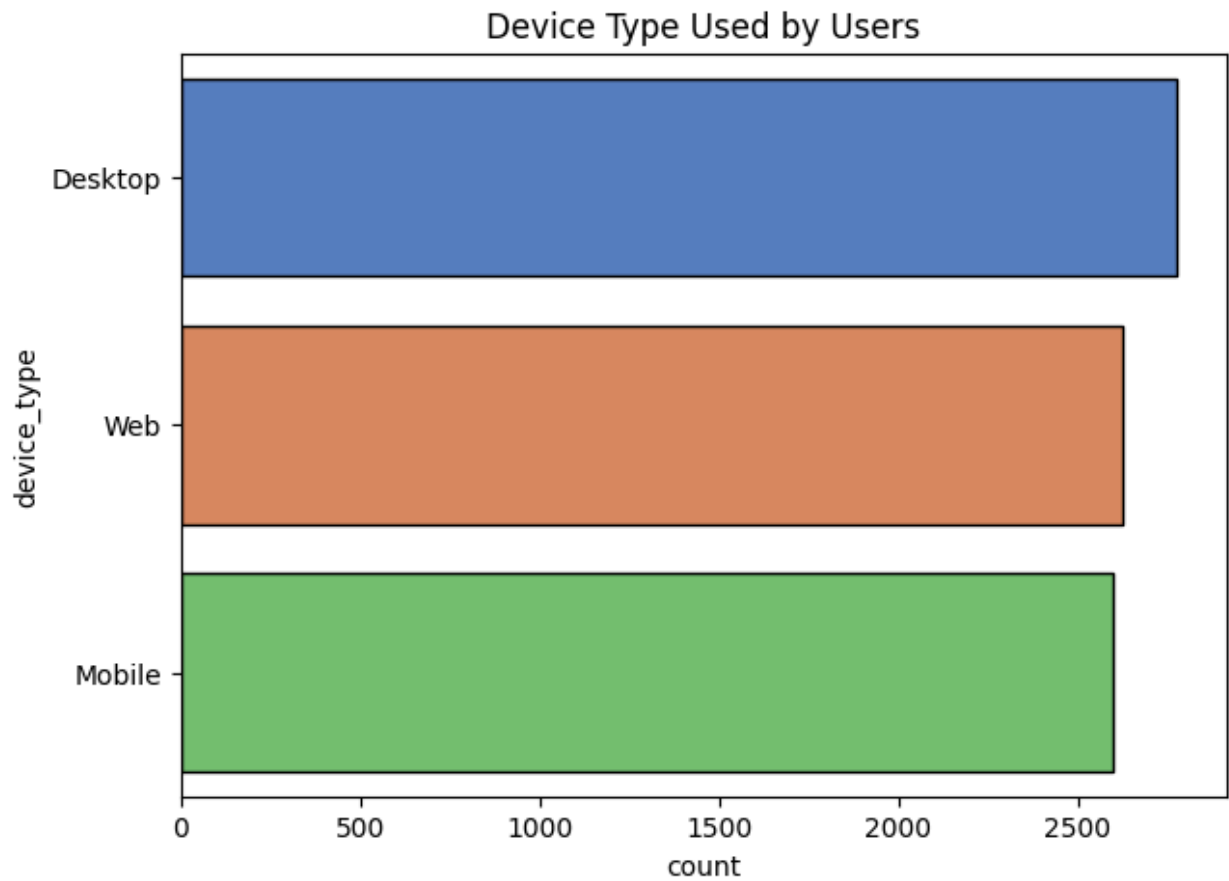


```
In [20]: plt.figure(figsize=(7,5))
sns.scatterplot(x="songs_played_per_day", y="skip_rate", hue="subscription_type")
plt.title("Songs Played per Day vs Skip Rate")
plt.show()
```

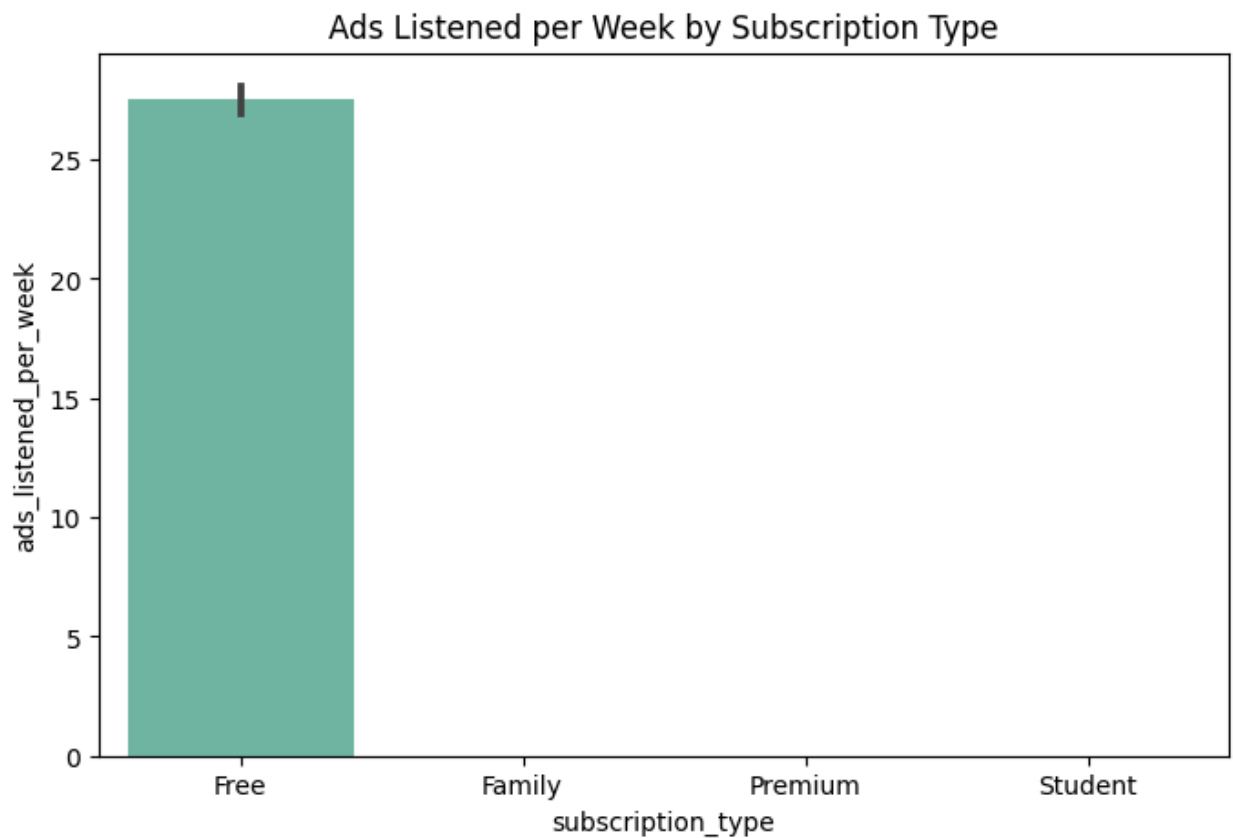


```
In [21]: plt.figure(figsize=(7,5))
sns.countplot(y="device_type", data=df, order=df["device_type"].value_counts())
```

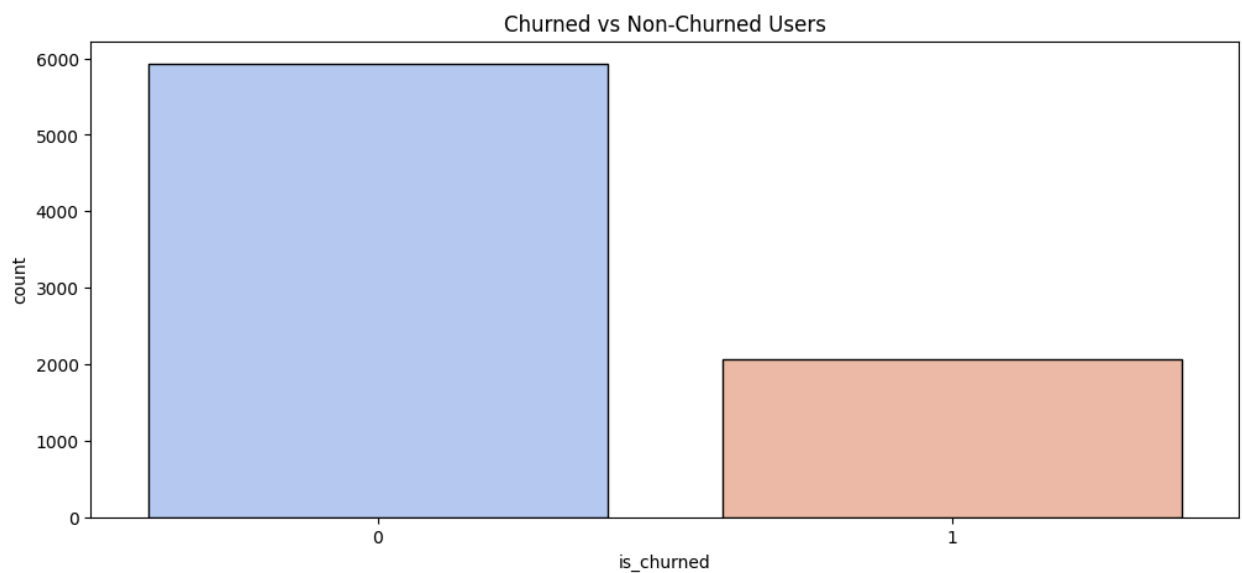
```
plt.title("Device Type Used by Users")  
plt.show()
```



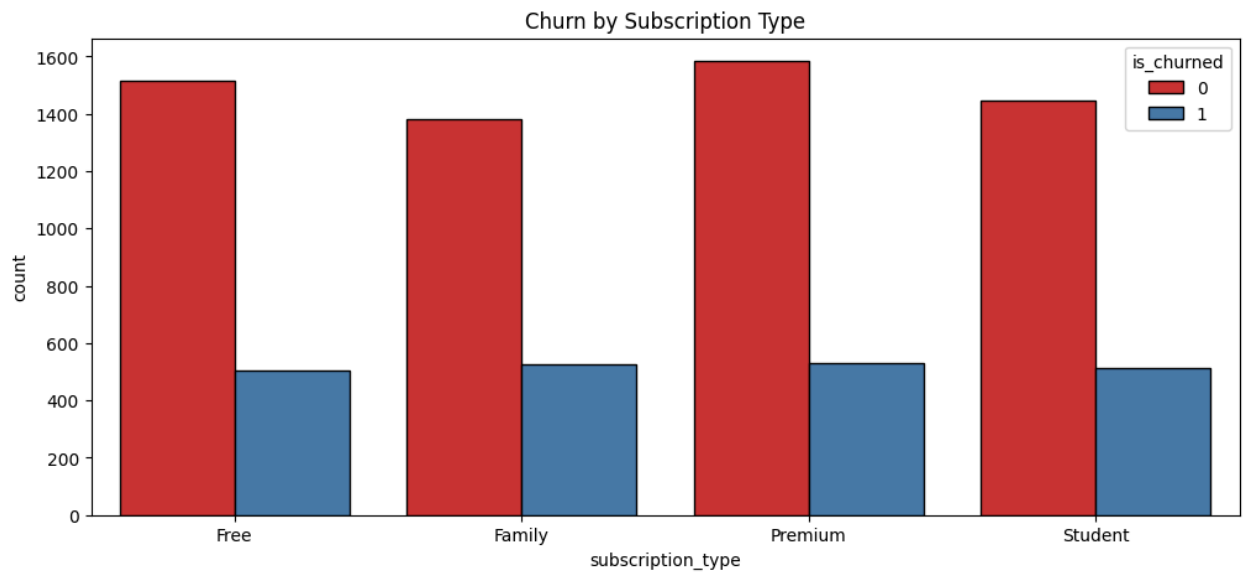
```
In [22]: plt.figure(figsize=(8,5))  
sns.barplot(x="subscription_type", y="ads_listened_per_week", data=df, palette  
plt.title("Ads Listened per Week by Subscription Type")  
plt.show()
```

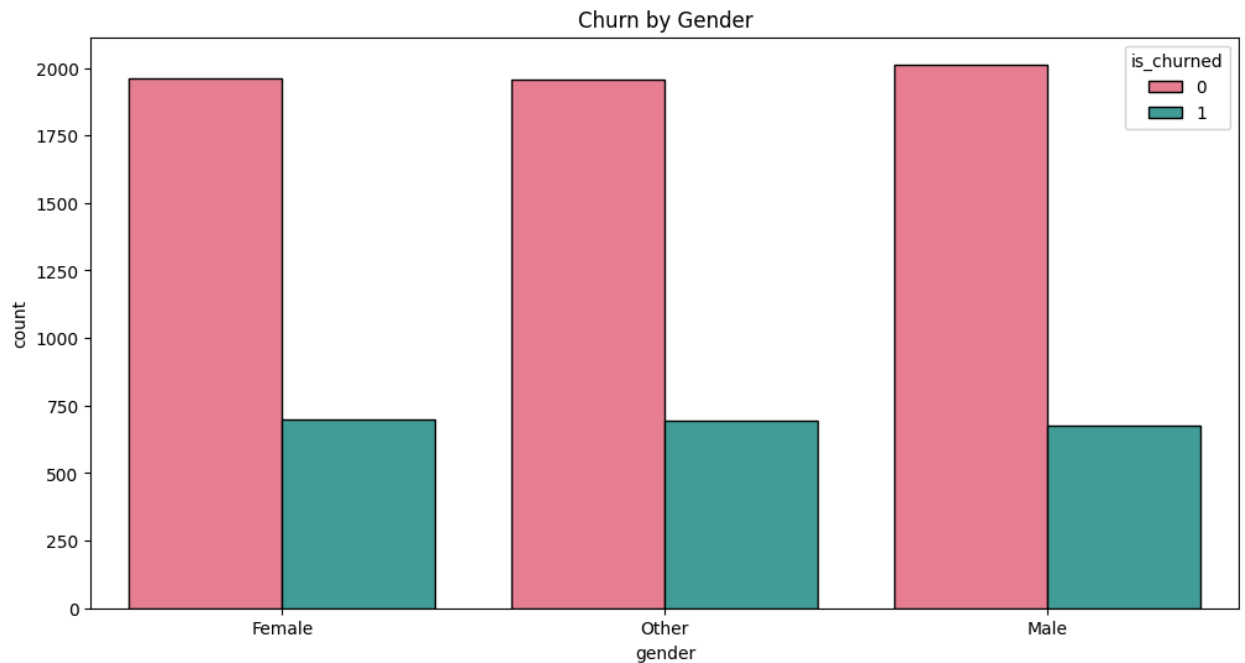
```
In [23]: plt.figure(figsize=(12,5))
sns.countplot(x="is_churned", data=df, palette="coolwarm", edgecolor = "black")
plt.title("Churned vs Non-Churned Users")
plt.show()
```



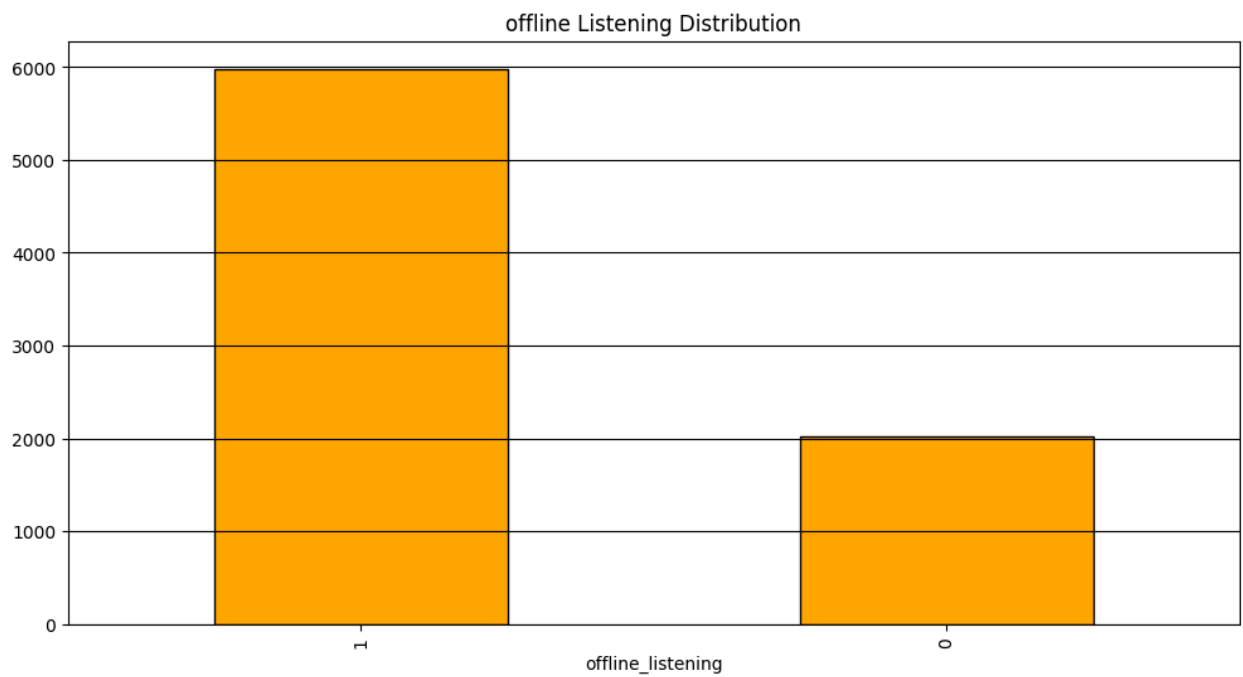
```
In [24]: plt.figure(figsize=(12,5))
sns.countplot(x="subscription_type", hue="is_churned", data=df, palette="Set1")
plt.title("Churn by Subscription Type")
plt.show()
```



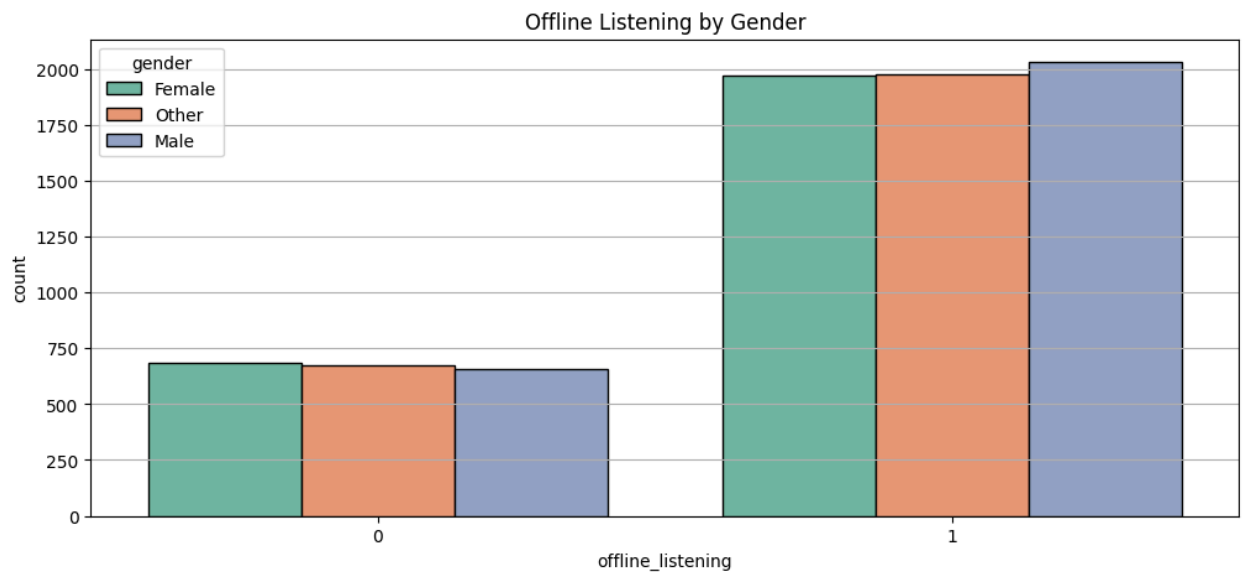
```
In [25]: plt.figure(figsize=(12,6))
sns.countplot(x="gender", hue="is_churned", data=df, palette="husl", edgecolor=
plt.title("Churn by Gender")
plt.show()
```



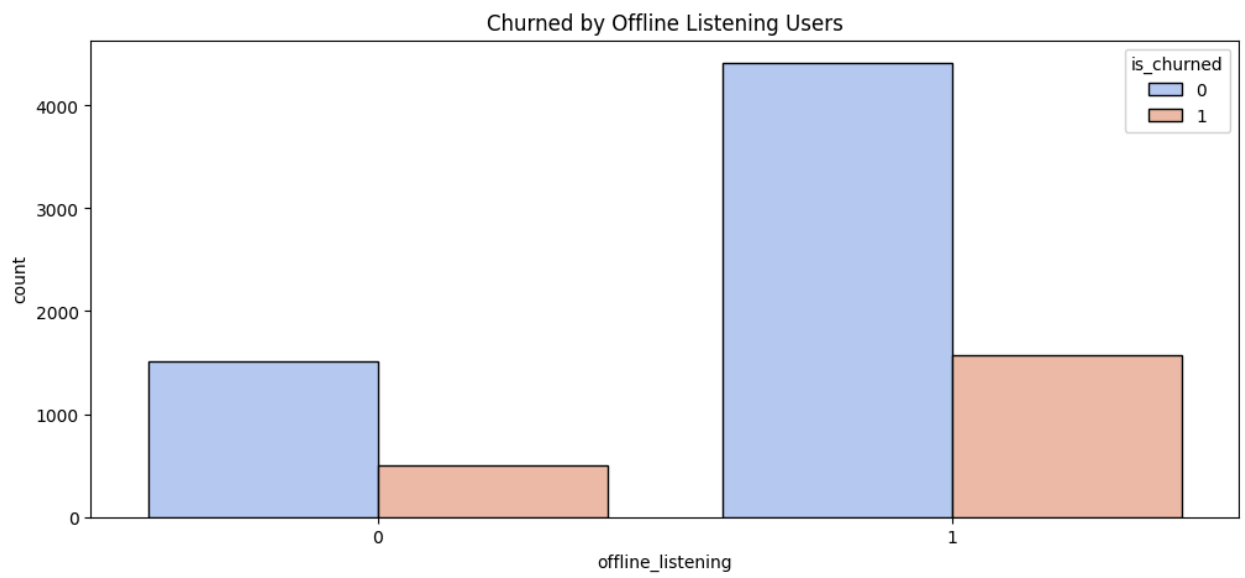
```
In [26]: plt.figure(figsize=(12,6))
df["offline_listening"].value_counts().plot(kind = "bar", color = "orange", ec
plt.grid(axis = "y", color = "black")
plt.title("offline Listening Distribution")
plt.show()
```



```
In [27]: plt.figure(figsize=(12,5))
sns.countplot(data=df, x="offline_listening", hue="gender", palette="Set2", ec
plt.title("Offline Listening by Gender")
plt.grid(axis = "y")
plt.show()
```



```
In [28]: plt.figure(figsize=(12,5))
sns.countplot(x="offline_listening", data=df, hue = "is_churned", palette="coo
plt.title("Churned by Offline Listening Users")
plt.show()
```



In []: